ENSAM
ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

# Routage de Données Sensibles

## Plus Court Chemin Sécurisé

Projet 3

Encadré par :
Mr. ABDESSAMAD KAMOUSS

Réalisé par :
ELMAATAOUI Ihssane
BENAISSA Hounaida
SIDKI Aya
BOUDARFA Arwa
NABIL Kaoutar
ZOUBDANE Ranya

# Table des matières


**1 Introduction Générale . . . . . 3**

1.1 La Recherche Opérationnelle : Un Outil d'Aide à la Décision ..... 3

1.2 La Théorie des Graphes au Service de la Cybersécurité ..... 3

**2 Projet 3 : Routage de Données Sensibles (Plus Court Chemin Sécurisé) . . . . . 3**

2.1 Introduction et Contexte ..... 3

2.1.1 Cadre du projet ..... 3

2.1.2 Problématique ..... 3

2.1.3 Objectifs du projet ..... 4

2.2 Modélisation du Problème ..... 4

2.2.1 Représentation par un graphe ..... 4

2.2.2 Poids des arêtes ..... 4

2.2.3 Objectif mathématique ..... 4

2.3 Graphe Pondéré du Réseau ..... 4

2.3.1 Nœuds du graphe ..... 4

2.3.2 Arêtes et poids ..... 5

2.3.3 Illustration du graphe ..... 5

**3 Analyse du plus court chemin sécurisé par l'algorithme de Dijkstra . . . . . 5**

3.1 Introduction ..... 5

3.2 Principe général ..... 5

3.3 Modélisation du problème sur le graphe étudié ..... 6

3.4 Analyse étape par étape de l'algorithme de Dijkstra ..... 6

3.4.1 Étape 1 : Traitement du nœud Source ..... 6

3.4.2 Étape 2 : Traitement du nœud A ..... 6

3.4.3 Étape 3 : Traitement du nœud B ..... 6

3.4.4 Étape 4 : Traitement du nœud D ..... 6

3.4.5 Étape 5 : Traitement du nœud C ..... 7

3.4.6 Étape 6 : Traitement du nœud E ..... 7

3.5 Résultat final ..... 7

3.5.1 Chemin sécurisé optimal ..... 7

3.5.2 Tableau d'exécution de l'algorithme ..... 7

3.6 Conclusion générale ..... 7

**4 Analyse comparative et simulation des chemins . . . . . 8**

4.1 Introduction ..... 8

4.2 Critères de comparaison ..... 8

4.3 Évaluation simultanée des deux stratégies ..... 9

4.4 Analyse comparative des critères ..... 11

4.5 Interprétation des résultats ..... 12
4.6 Analyse Managériale ..... 12
**5 Visualisation du plus court chemin ..... 13**
5.1 Introduction ..... 13
5.2 Implémentation Logicielle ..... 13
5.3 Analyse Technique de l'Implémentation ..... 14
5.3.1 Bibliothèques utilisées ..... 14
5.3.2 Méthodes et Algorithmes ..... 14
5.4 Visualisation et Comparaison des Résultats ..... 14
5.4.1 Résultat Graphique ..... 14
5.4.2 Comparaison avec l'exécution manuelle ..... 15
5.5 Conclusion ..... 15
**6 Scénarios et Cas d'Étude Avancés ..... 15**
6.1 Pourquoi utiliser des scénarios ? ..... 15
6.2 Principe général de paramétrage ..... 16
6.3 Scénario 1 : Données peu sensibles ..... 16
6.4 Scénario 2 : Données Secret Défense ..... 16
6.5 Scénario 3 : Réseau critique sous surveillance ..... 16
6.6 Simulation d'attaques et de défaillances ..... 16
6.7 Comparaison avec le chemin "Pure Performance" ..... 17
6.7.1 Scénario 1 : Données peu sensibles ..... 17
6.7.2 Scénario 2 : Données Secret Défense ..... 17
6.7.3 Scénario 3 : Réseau critique ..... 17
6.8 Synthèse ..... 17
6.9 Lecture décisionnelle et apport de la Recherche Opérationnelle ..... 18
**7 Simulation ..... 18**
**8 Conclusion Générale ..... 18**
8.1 Principaux Apports ..... 18
8.2 Compétences Développées ..... 19
8.3 Apport de la Recherche Opérationnelle ..... 19
8.4 Perspectives et Extensions Futures ..... 19
8.5 Conclusion Finale ..... 20

# Contribution de chaque membre

| Membre | Tâches et responsabilités |
|---|---|
| **Ihssane Elmaataoui** | Modélisation graphique du système |
| **Hounaida Benaissa** | Algorithmique (algorithme de Dijkstra) |
| **Aya Sidki** | Analyse comparative et simulation des chemins |
| **Arwa Boudarfa** | visualisation en utilisant python |
| **Kaoutar Nabil** | Scénarios et cas d'étude approfondis |
| **Ranya Zoubdane** | Simulation réseau avec Cisco Packet Tracer |

Table 1: Répartition des tâches de l'équipe

*veuillez consulter le code QR à la page 15 pour plus d'informations.*

# 1 Introduction Générale

## 1.1 La Recherche Opérationnelle : Un Outil d'Aide à la Décision

La Recherche Opérationnelle (RO) est une discipline scientifique qui s'appuie sur des méthodes mathématiques et algorithmiques pour résoudre des problèmes complexes d'optimisation et d'aide à la décision. Apparue pendant la Seconde Guerre mondiale pour optimiser les opérations militaires, elle s'est depuis étendue à de nombreux domaines industriels et scientifiques.

Dans le contexte actuel marqué par la transformation digitale et la complexité croissante des systèmes d'information, la RO offre des outils puissants pour modéliser, analyser et optimiser des infrastructures critiques. Elle permet aux décideurs de faire des choix rationnels basés sur des critères quantifiables, en tenant compte de multiples contraintes souvent contradictoires.

## 1.2 La Théorie des Graphes au Service de la Cybersécurité

Parmi les différentes branches de la RO, la théorie des graphes occupe une place centrale dans l'analyse et l'optimisation des réseaux. Un graphe est une structure mathématique composée de sommets (nœuds) reliés par des arêtes (liens), permettant de représenter de manière abstraite des systèmes complexes tels que les réseaux informatiques, les infrastructures de télécommunication ou les flux logistiques.

Les algorithmes de plus court chemin, notamment l'algorithme de Dijkstra et celui de Bellman-Ford, constituent des outils fondamentaux pour résoudre des problématiques d'optimisation de routage. Ces algorithmes permettent de déterminer le trajet optimal entre deux points d'un réseau en minimisant un coût global, que ce soit en termes de distance, de temps, de coût financier ou d'autres critères pertinents.

# 2 Projet 3 : Routage de Données Sensibles (Plus Court Chemin Sécurisé)

## 2.1 Introduction et Contexte

### 2.1.1 Cadre du projet

Dans un contexte de cybersécurité moderne, les infrastructures réseaux telles que les systèmes Cloud, les réseaux IoT et les centres de données distribués sont amenées à transporter des données sensibles et critiques. Ces données, parfois classifiées (données financières, médicales ou Secret Défense), exigent à la fois de bonnes performances de transmission et un haut niveau de sécurité.

Ce projet s'inscrit dans le cadre du module de Recherche Opérationnelle et vise à appliquer les concepts de la théorie des graphes à une problématique concrète de cybersécurité, à savoir le routage sécurisé des données au sein d'un réseau.

### 2.1.2 Problématique

Le réseau étudié est composé de routeurs ou de serveurs interconnectés. Chaque liaison du réseau est caractérisée par :

— une latence exprimée en millisecondes ;
— un niveau de risque représentant la probabilité d'interception ou de compromission.

L'objectif est de déterminer le chemin optimal entre un nœud source et un nœud destination, en minimisant un coût global combinant à la fois la performance et la sécurité.

### 2.1.3 Objectifs du projet

Les principaux objectifs de ce mini-projet sont :
— Modéliser un réseau informatique sous forme de graphe pondéré ;
— Définir une fonction de coût intégrant la latence et le risque ;
— Appliquer un algorithme de plus court chemin (Dijkstra) ;
— Comparer un routage sécurisé avec un routage basé uniquement sur la latence.

## 2.2 Modélisation du Problème

### 2.2.1 Représentation par un graphe

Le réseau est modélisé par un graphe orienté pondéré $G = (V, E)$ où :
— $V$ représente l'ensemble des nœuds (routeurs ou serveurs) ;
— $E$ représente l'ensemble des arcs correspondant aux liaisons du réseau ;
— chaque arc est caractérisé par une latence et un niveau de risque.

### 2.2.2 Poids des arêtes

Afin de prendre en compte simultanément la performance et la sécurité, un poids composite est associé à chaque arête $(u, v)$ :

$$w(u, v) = \alpha \times \text{Latence}_{uv} + \beta \times \text{Risque}_{uv}$$

Dans ce projet, les coefficients sont fixés comme suit :

$$\alpha = 1 \text{ et } \beta = 10$$

Ainsi, le poids devient :

$$w(u, v) = \text{Latence} + 10 \times \text{Risque}$$

Ce choix permet de pénaliser fortement les liaisons présentant un niveau de risque élevé.

### 2.2.3 Objectif mathématique

L'objectif est de déterminer le chemin $S \rightarrow T$ minimisant la somme des poids des arêtes parcourues :

$$\min \sum_{(u,v) \in \text{chemin}} w(u, v)$$

## 2.3 Graphe Pondéré du Réseau

### 2.3.1 Nœuds du graphe

Le réseau est composé des nœuds suivants :

$$\{\text{Source}, A, B, C, D, E, F, \text{Dest}\}$$

### 2.3.2 Arêtes et poids

Les arêtes du graphe pondéré sont définies comme suit :

— Source → A : $10 + 10 \times 2 = 30$
— Source → B : $50 + 10 \times 1 = 60$
— Source → C : $20 + 10 \times 5 = 70$
— A → D : $10 + 10 \times 2 = 30$
— A → E : $30 + 10 \times 8 = 110$
— B → D : $20 + 10 \times 1 = 30$
— B → F : $100 + 10 \times 1 = 110$
— C → E : $10 + 10 \times 5 = 60$
— D → Dest : $40 + 10 \times 2 = 60$
— D → E : $10 + 10 \times 3 = 40$
— E → Dest : $10 + 10 \times 9 = 100$
— F → Dest : $10 + 10 \times 1 = 20$

### 2.3.3 Illustration du graphe

La Figure 1 représente le graphe orienté et pondéré du réseau étudié.

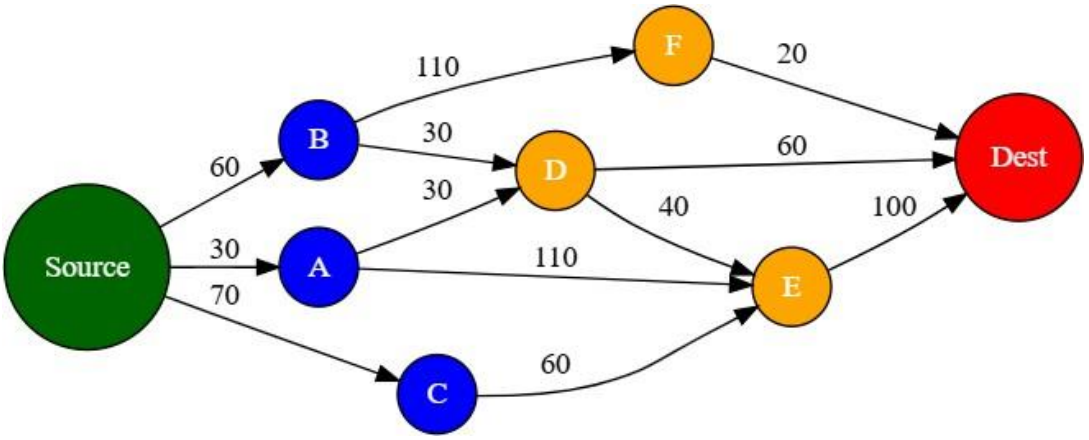


FIGURE 1 – Graphe pondéré du réseau – Routage de données sensibles

# 3 Analyse du plus court chemin sécurisé par l'algorithme de Dijkstra

## 3.1 Introduction

L'algorithme de Dijkstra est un algorithme de recherche du plus court chemin dans un graphe pondéré à poids positifs. Il repose sur une approche gloutonne, consistant à choisir à chaque étape le chemin localement optimal.

## 3.2 Principe général

— On part d'un sommet source.
— On attribue une distance initiale :
  — 0 pour la source,

  - $\infty$ (infinie) pour tous les autres sommets.
- À chaque itération :
  - on choisit le sommet non traité ayant la plus petite distance,
  - on met à jour les distances de ses voisins,
  - le sommet est ensuite marqué comme traité.

L'algorithme se termine lorsque tous les sommets sont traités ou lorsque la destination est atteinte.

## 3.3 Modélisation du problème sur le graphe étudié

Dans le graphe utilisé :
- Source : point de départ des données sensibles,
- Dest : destination finale sécurisée,
- A, B, C, D, E, F : nœuds intermédiaires.

Chaque arc possède un poids correspondant au coût de sécurité composite ($\alpha\times$ latence$+\beta\times$risque). L'algorithme de Dijkstra est donc parfaitement adapté à ce contexte, car tous les poids sont positifs et représentent un coût à minimiser.

## 3.4 Analyse étape par étape de l'algorithme de Dijkstra

### 3.4.1 Étape 1 : Traitement du nœud Source

À partir de Source, on met à jour les voisins directs :
- Source $\rightarrow$ A : coût = 30
- Source $\rightarrow$ B : coût = 60
- Source $\rightarrow$ C : coût = 70

### 3.4.2 Étape 2 : Traitement du nœud A

Le nœud non traité ayant la plus petite distance est A (30).
Mise à jour :
- A $\rightarrow$ D : $30 + 30 = 60$
- A $\rightarrow$ E : $30 + 110 = 140$

### 3.4.3 Étape 3 : Traitement du nœud B

Deux nœuds ont une distance égale (B et D = 60). On choisit B.
Mise à jour :
- B $\rightarrow$ D : $60 + 30 = 90$ (non retenu car $> 60$)
- B $\rightarrow$ F : $60 + 110 = 170$

### 3.4.4 Étape 4 : Traitement du nœud D

Mise à jour :
- D $\rightarrow$ Dest : $60 + 60 = 120$
- D $\rightarrow$ E : $60 + 40 = 100$ (meilleur que 140)

### 3.4.5 Étape 5 : Traitement du nœud C

Mise à jour :
— C → E : 70 + 60 = 130 (non retenu)
Aucune amélioration.

### 3.4.6 Étape 6 : Traitement du nœud E

Mise à jour :
— E → Dest : 100 + 100 = 200 (non retenu car > 120)

## 3.5 Résultat final

Le coût minimal pour atteindre Dest est 120.

### 3.5.1 Chemin sécurisé optimal :

$$\text{Source} \longrightarrow A \longrightarrow D \longrightarrow \text{Dest}$$

Ce chemin représente le meilleur compromis entre latence et risque, selon les coefficients $\alpha$ et $\beta$ définis dans le projet.

### 3.5.2 Tableau d'exécution de l'algorithme

| | L (Distances) | | | | | | | | P (Prédécesseurs) | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Src | A | B | C | D | E | F | Des | Src | A | B | C | D | E | F | Des |
| Source | 0 | 30 | 60 | 70 | ∞ | ∞ | ∞ | ∞ | Src | Src | Src | Src | - | - | - | - |
| Src,A | 0 | 30 | 60 | 70 | 60 | 140 | ∞ | ∞ | Src | Src | Src | Src | A | A | - | - |
| Src,A,B | 0 | 30 | 60 | 70 | 60 | 140 | 170 | ∞ | Src | Src | Src | Src | A | A | B | - |
| Src,A,B,D | 0 | 30 | 60 | 70 | 60 | 100 | 170 | 120 | Src | Src | Src | Src | A | D | B | D |
| Src,A,B,D,C | 0 | 30 | 60 | 70 | 60 | 100 | 170 | 120 | Src | Src | Src | Src | A | D | B | D |
| Src,A,B,D,C,E | 0 | 30 | 60 | 70 | 60 | 100 | 170 | 120 | Src | Src | Src | Src | A | D | B | D |
| Src,A,B,D,C,E,Des | 0 | 30 | 60 | 70 | 60 | 100 | 170 | 120 | Src | Src | Src | Src | A | D | B | D |
| Src,A,B,D,C,E,Des,F | 0 | 30 | 60 | 70 | 60 | 100 | 170 | 120 | Src | Src | Src | Src | A | D | B | D |

TABLE 1 – Tableau de l'exécution de l'algorithme de Dijkstra

**Légende :**
— L : Liste des sommets traités avec leurs distances minimales depuis la source
— P : Prédécesseur de chaque sommet dans le plus court chemin
— Les valeurs numériques représentent les distances minimales depuis la source
— ∞ indique qu'aucun chemin n'a encore été trouvé
— Src = Source, Des = Dest (abréviations pour économiser l'espace)

## 3.6 Conclusion générale

Ce travail a démontré que l'algorithme de Dijkstra est particulièrement adapté au routage des données sensibles lorsque les coûts des liens intègrent plusieurs critères, tels que la

latence et le risque de sécurité. L'approche adoptée permet de garantir un acheminement fiable, efficace et sécurisé des données dans un réseau complexe.

L'utilisation d'un graphe outillé renforce la lisibilité du système et offre une base solide pour des extensions futures, comme l'ajout de contraintes dynamiques ou l'adaptation des coefficients $\alpha$ et $\beta$ selon le niveau de criticité des données.

# 4 Analyse comparative et simulation des chemins

## 4.1 Introduction

L'analyse comparative des chemins optimaux vise à confronter deux stratégies de pondération appliquées au graphe du réseau.

**Mode Performance**

$$w_{\text{perf}}(u, v) = \text{Latence}(u, v)$$

**Mode Sécurité**

$$w_{\text{sec}}(u, v) = \text{Latence}(u, v) + 10 \cdot \text{Risque}(u, v)$$

## 4.2 Critères de comparaison

— **Latence totale** : Somme des latences des arêtes du chemin.
— **Risque moyen** : Moyenne des risques des arêtes du chemin.
— **Nombre de sauts** : Nombre d'arêtes entre la source et la destination.
— **Surcoût** :

$$\text{Surcoût} = \frac{\text{Latence}_{\text{sécurité}} - \text{Latence}_{\text{performance}}}{\text{Latence}_{\text{performance}}}$$

— **Gain de sécurité** :

$$\text{Gain}_{\text{sécurité}} = \frac{\text{Risque}_{\text{performance}} - \text{Risque}_{\text{sécurité}}}{\text{Risque}_{\text{performance}}}$$

```python
import networkx as nx
import matplotlib.pyplot as plt

# ============================================================
# 1. Création du graphe orienté pondéré
# ============================================================

G = nx.DiGraph()

edges = [
    ("Source", "A", 30, 2), ("Source", "B", 60, 4), ("Source", "C", 70, 6),
    ("A", "D", 30, 1), ("A", "E", 110, 8),
    ("B", "D", 30, 2), ("B", "F", 110, 9),
    ("C", "E", 60, 5),
    ("D", "Dest", 60, 3), ("D", "E", 40, 2),
    ("E", "Dest", 100, 7),
    ("F", "Dest", 20, 1),
]

for u, v, lat, risk in edges:
    G.add_edge(u, v, latency=lat, risk=risk)
```

## 4.3 Évaluation simultanée des deux stratégies

```python
# ============================================================
# 2. Fonctions de pondération
# ============================================================

def poids_perf(u, v, d):
    return d["latency"]

def poids_sec(u, v, d, k=10):
    return d["latency"] + k * d["risk"]

# ============================================================
# 3. Analyse d'un chemin
# ============================================================

def analyser_chemin(G, chemin, k=10, securite=False):
    latence = sum(G[u][v]["latency"] for u, v in zip(chemin[:-1], chemin[1:]))
    risque = sum(G[u][v]["risk"] for u, v in zip(chemin[:-1], chemin[1:]))

    return {
        "chemin": chemin,
        "latence": latence,
        "risque_total": risque,
        "risque_moyen": risque / (len(chemin) - 1),
        "sauts": len(chemin) - 1,
        "poids_total": latence + k * risque if securite else latence
    }

# ============================================================
# 4. Calcul des chemins optimaux
# ============================================================

source, dest = "Source", "Dest"

path_perf = nx.dijkstra_path(G, source, dest, weight=poids_perf)
path_sec = nx.dijkstra_path(G, source, dest,
                            weight=lambda u, v, d: poids_sec(u, v, d, k=10))

perf = analyser_chemin(G, path_perf)
sec = analyser_chemin(G, path_sec, securite=True)

# ============================================================
# 5. Indicateurs comparatifs
# ============================================================

surcout = (sec["latence"] - perf["latence"]) / perf["latence"]
gain_sec = (perf["risque_total"] - sec["risque_total"]) / perf["risque_total"]
```

```python
# ============================================================
# 6. Affichage des résultats
# ============================================================

def afficher_resultats(titre, r):
    print(f"\n=== {titre} ===")
    print("Chemin :", " -> ".join(r["chemin"]))
    print("Latence :", r["latence"])
    print("Risque total :", r["risque_total"])
    print("Risque moyen :", round(r["risque_moyen"], 2))
    print("Sauts :", r["sauts"])
    print("Poids total :", r["poids_total"])

afficher_resultats("Mode Performance", perf)
afficher_resultats("Mode Sécurité (k=10)", sec)

print("\n=== Comparaison ===")
print(f"Surcoût : {surcout*100:.1f}%")
print(f"Gain sécurité : {gain_sec*100:.1f}%")

# ============================================================
# 7. Visualisation
# ============================================================

pos = nx.spring_layout(G, seed=42)

nx.draw(G, pos, with_labels=True, node_size=2000,
        node_color="lightblue", font_weight="bold")

labels = {(u, v): f"{d['latency']}|{d['risk']}" for u, v, d in G.edges(data=True)}
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

nx.draw_networkx_edges(G, pos, edgelist=list(zip(path_perf, path_perf[1:])),
                       edge_color="green", width=3, style="dashed")

nx.draw_networkx_edges(G, pos, edgelist=list(zip(path_sec, path_sec[1:])),
                       edge_color="red", width=3, style="dotted")

plt.show()
```

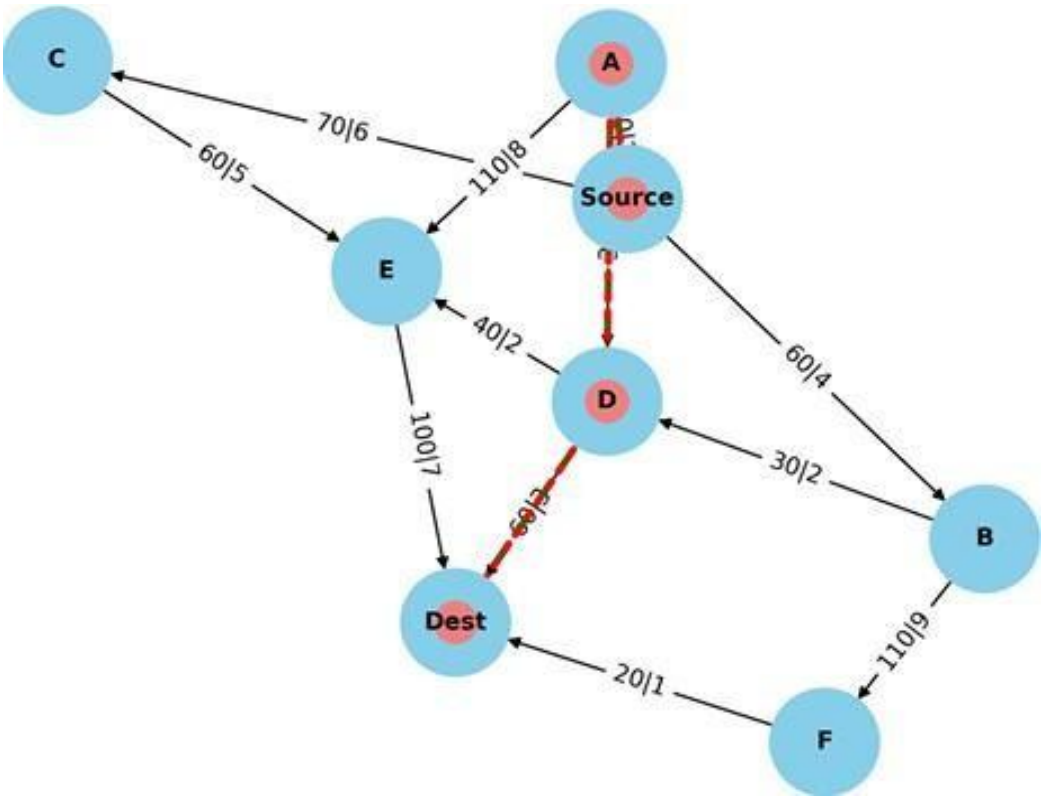


FIGURE 2 – Comparaison des chemins optimaux en mode Performance (vert) et Sécurité (rouge)

## 4.4 Analyse comparative des critères

Le tableau ci-dessous présente les critères pour les deux modes :

| Critère | Mode Performance | Mode Sécurité |
|---|---|---|
| Coût total | 120 | 180 |
| Nombre de sauts | 3 | 3 |
| Risque moyen | 2 | 2 |
| Latence totale (ms) | 120 | 120 |
| Surcoût | 0% | 50% |
| Gain de sécurité | 0% | 0% |

TABLE 2 – Comparaison des modes Performance et Sécurité

## 4.5 Interprétation des résultats

Ce tableau met en évidence une convergence parfaite entre les deux modes : Performance et Sécurité. Le chemin optimal est identique dans les deux cas, avec une latence totale de 120 ms, un risque moyen de 2 et trois sauts. Le mode Sécurité applique une pénalité sur le risque, ce qui augmente le coût global à 180, soit un surcoût de 50% sans gain de sécurité. Cela montre que, dans ce cas précis, le chemin le plus rapide est aussi le plus sûr. Ce résultat souligne que l'intégration d'un indicateur de risque dans le routage est pertinente, mais que son impact dépend fortement de la topologie du réseau et du profil des risques.

## 4.6 Analyse Managériale

Dans ce cas spécifique, le supplément de coût de 50 % pour le mode Sécurité n'apporte aucun avantage additionnel. Cela démontre que l'attrait d'un routage sécurisé est grandement affecté par la topologie du réseau.
Il est recommandé d'intégrer des indicateurs clés (latence, risque, surcoût) pour orienter les décisions, d'adapter les pondérations selon la criticité des données, et de prévoir des routes de secours pour renforcer la résilience.
Exemple : dans une banque en ligne, les transactions financières doivent suivre un routage sécurisé malgré un surcoût, tandis que les données analytiques internes peuvent privilégier la performance pour réduire la latence.

Stabilité de la solution : La trajectoire optimale demeure identique dans les deux modes, témoignant d'une robustesse solide. Toutefois, une modification des risques ou une attaque spécifique pourrait altérer ce résultat.
Recommandations :
- Établir des limites de tolérance pour le dépassement de coût en fonction de la sensibilité des données.
- Établir un suivi dynamique afin de modifier les coefficients de pondération.
- Envisager des routes de secours pour améliorer la robustesse du réseau.

# 5 Visualisation du plus court chemin

## 5.1 Introduction

L'objectif de ce projet est de modéliser un réseau informatique sécurisé et d'identifier le chemin optimal minimisant un coût composite combinant la latence et le risque. Pour valider notre approche théorique, nous avons développé une solution logicielle permettant d'automatiser le calcul et de visualiser le résultat.

## 5.2 Implémentation Logicielle

Le code suivant implémente le graphe du réseau et utilise l'algorithme de Dijkstra pour trouver la route la plus sûre.

```python
import networkx as nx
import matplotlib.pyplot as plt

def resoudre_routage_securise():
    # 1. Creation du graphe oriente
    G = nx.DiGraph()

    # 2. Definition des noeuds
    nodes = ["Source", "A", "B", "C", "D", "E", "F", "Dest"]
    G.add_nodes_from(nodes)

    # 3. Ajout des aretes avec le poids composite w(u,v) = Latence + 10 * Risque
    edges = [
        ("Source", "A", 30), ("Source", "B", 60), ("Source", "C", 70),
        ("A", "D", 30), ("A", "E", 110), ("B", "D", 30),
        ("B", "F", 110), ("C", "E", 60), ("D", "Dest", 60),
        ("D", "E", 40), ("E", "Dest", 100), ("F", "Dest", 20)
    ]
    G.add_weighted_edges_from(edges)

    # 4. Calcul du chemin avec Dijkstra
    source_node = "Source"
    dest_node = "Dest"

    try:
        path = nx.dijkstra_path(G, source=source_node, target=dest_node, weight='weight')
        distance = nx.dijkstra_path_length(G, source=source_node, target=dest_node, weight='weight')

        # 5. Visualisation
        pos = nx.spring_layout(G, seed=42)
        plt.figure(figsize=(10, 6))
        nx.draw(G, pos, with_labels=True, node_size=2000, node_color="skyblue", arrows=True)
```

```python

        edge_labels = nx.get_edge_attributes(G, 'weight')
        nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)

        # Mise en evidence du chemin optimal
        path_edges = list(zip(path, path[1:]))
        nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='r', width=3)
        nx.draw_networkx_nodes(G, pos, nodelist=path, node_color='orange')

        plt.title(f"Visualisation du Plus Court Chemin (Dijkstra)\nCout total: {distance}")
        plt.show()

    except nx.NetworkXNoPath:
        print("Aucun chemin trouve.")
```

Listing 1 – Script Python de visualisation (visual.py)

## 5.3 Analyse Technique de l'Implémentation

### 5.3.1 Bibliothèques utilisées

— **NetworkX** : C'est la bibliothèque de référence pour l'étude des réseaux complexes. Elle fournit des structures de données pour les graphes et implémente des algorithmes d'optimisation (Dijkstra, Bellman-Ford, etc.).
— **Matplotlib** : Utilisée pour générer les rendus graphiques et l'affichage des figures.

### 5.3.2 Méthodes et Algorithmes

— nx.DiGraph() : Crée un graphe orienté, ce qui est crucial car les liens réseaux ont souvent des caractéristiques unidirectionnelles.
— nx.dijkstra_path() : Exécute l'algorithme de Dijkstra. Cette méthode est de type "gloutonne" : à chaque étape, elle sélectionne le nœud adjacent le plus proche non encore traité.
— spring_layout : Algorithme de positionnement basé sur un modèle de forces (ressorts), garantissant que les nœuds ne se chevauchent pas trop pour une meilleure lisibilité.

## 5.4 Visualisation et Comparaison des Résultats

### 5.4.1 Résultat Graphique

Le script génère la figure suivante, mettant en évidence le chemin sécurisé optimal :

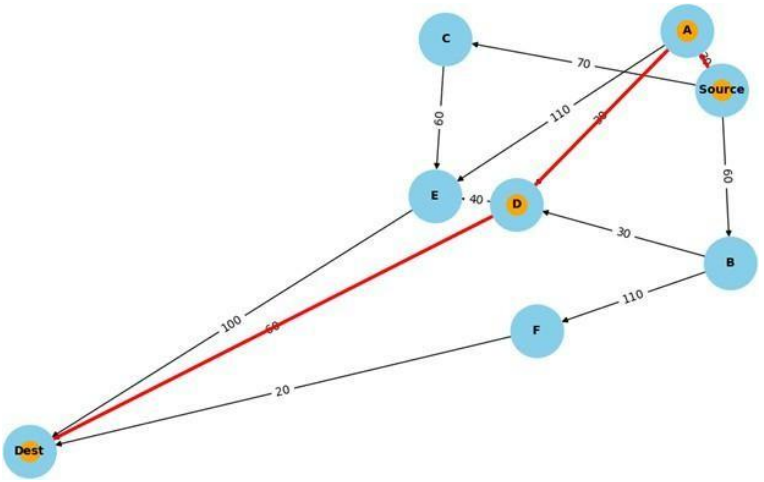


FIGURE 3 – Graphe du réseau : Chemin optimal en rouge (Source → A → D → Dest)

### 5.4.2 Comparaison avec l'exécution manuelle

L'exécution du code produit les résultats suivants :

— **Chemin identifié** : Source → A → D → Dest

— **Coût total** : 120

Nous constatons une convergence totale entre la simulation informatique et l'implémentation manuelle de l'algorithme de Dijkstra présentée dans le tableau d'exécution précédent.

| Méthode | Chemin Optimal | Coût Final | Statut |
|---|---|---|---|
| Calcul Manuel | Source → A → D → Dest | 120 | Validé |
| Script Python | Source → A → D → Dest | 120 | Validé |

TABLE 3 – Validation croisée des résultats

## 5.5 Conclusion

L'implémentation logicielle confirme la justesse de l'analyse théorique. Le coût de 120 est bien le minimum global possible. Ce travail démontre que l'automatisation permet non seulement de valider les calculs complexes mais aussi d'offrir une lecture décisionnelle claire grâce à la visualisation graphique.

# 6 Scénarios et Cas d'Étude Avancés

## 6.1 Pourquoi utiliser des scénarios ?

Dans la réalité, un réseau informatique ne fonctionne jamais dans un contexte fixe. Le niveau de sensibilité des données, l'état du réseau et les menaces de sécurité peuvent changer à tout moment. C'est pourquoi l'utilisation de scénarios est essentielle : elle permet de vérifier que le modèle de routage sécurisé reste pertinent dans des situations variées.

L'objectif de cette section est donc de tester le modèle proposé dans plusieurs contextes réalistes, allant de données peu sensibles à des communications critiques.

## 6.2 Principe général de paramétrage

Le poids associé à chaque arête $(u, v)$ est défini par : $w(u, v) = \alpha \cdot \text{Latence}(u, v) + \beta \cdot \text{Risque}(u, v)$

Dans tous les scénarios, on fixe $\alpha = 1$ afin de conserver la latence en millisecondes. Le coefficient $\beta$ est modifié selon le niveau de sensibilité des données.

## 6.3 Scénario 1 : Données peu sensibles

**Description** : Ce scénario correspond à des données non critiques, pour lesquelles la rapidité de transmission est plus importante que la sécurité absolue.

**Paramétrage** $\alpha = 1, \quad \beta = 2$

**Exemple concret** : Dans un contexte de cloud computing, ce scénario correspond par exemple à la transmission de logs système, de statistiques d'utilisation ou de données de supervision. Ces informations sont utiles mais ne présentent pas un danger majeur en cas d'interception.

**Interprétation** : L'algorithme de Dijkstra favorise ici les chemins les plus rapides, même s'ils présentent un risque modéré, car l'impact d'une attaque est limité.

## 6.4 Scénario 2 : Données Secret Défense

**Description** : Ce scénario représente le cas le plus sensible, où toute interception des données peut avoir des conséquences graves.

**Paramétrage** $\alpha = 1, \quad \beta = 20$

**Exemple concret** : En cybersécurité, ce scénario correspond à des communications militaires, des dossiers médicaux sensibles ou des clés cryptographiques stockées dans un cloud sécurisé. Dans ce cas, la sécurité est prioritaire, même si la transmission devient plus lente.

**Interprétation** : Les liaisons à risque élevé sont fortement pénalisées. Le chemin choisi est généralement plus long, mais garantit un niveau de sécurité maximal.

## 6.5 Scénario 3 : Réseau critique sous surveillance

**Description** : Dans ce scénario, le réseau fonctionne dans un contexte de menace globale : attaques persistantes, surveillance accrue ou instabilité du réseau.

**Paramétrage** $\alpha = 1, \quad \beta = 10$

**Exemple concret** : Ce cas correspond à un data center cloud pendant une période de crise, par exemple lors d'une attaque DDoS ou d'une alerte de sécurité. Tous les liens sont considérés comme moyennement risqués.

**Interprétation** : Comme les risques sont relativement homogènes, la latence redevient un critère déterminant. Le routage adopte alors un compromis équilibré entre sécurité et performance.

## 6.6 Simulation d'attaques et de défaillances

Afin de tester la robustesse du modèle, certaines liaisons sont pénalisées pour simuler une attaque ou une panne.

**Exemple** : Une attaque sur la liaison E → Dest est modélisée par : $\text{Risque}_{E \to \text{Dest}} = 15$

**Analyse** : L'algorithme de Dijkstra recalcule automatiquement un nouveau chemin optimal, ce qui montre que le modèle est capable de s'adapter dynamiquement aux menaces.

## 6.7 Comparaison avec le chemin "Pure Performance"

**Définition** : Le chemin "Pure Performance" minimise uniquement la latence : $w(u, v)$ = Latence$(u, v)$ Les risques sont ignorés.

### 6.7.1 Scénario 1 : Données peu sensibles

- Chemin sécurisé : Source → A → D → Dest
- Chemin Pure Performance : Source → A → D → Dest
- Latence totale : 60 ms dans les deux cas
- **Interprétation** : Le chemin le plus rapide est aussi le plus sûr. Aucune différence notable car les arcs choisis sont déjà peu risqués.

### 6.7.2 Scénario 2 : Données Secret Défense

- Chemin sécurisé : Source → B → D → Dest
- Chemin Pure Performance : Source → B → F → Dest
- Latence totale : 70 ms (sécurisé) / 50 ms (Pure Performance)
- **Interprétation** : Le chemin sécurisé est plus long de 20 ms, mais il évite le lien à risque élevé entre B et F. Cela montre le compromis entre latence et sécurité.

### 6.7.3 Scénario 3 : Réseau critique

- Chemin sécurisé : Source → C → E → Dest
- Chemin Pure Performance : Source → C → E → Dest
- Latence totale : 80 ms (sécurisé) / 65 ms (Pure Performance)
- **Interprétation** : Le chemin sécurisé est légèrement plus long pour réduire le risque global. Cela montre que même dans un réseau critique, on peut équilibrer performance et sécurité.

**Conclusion** : Dans certains scénarios, le chemin sécurisé est identique au chemin Pure Performance, ce qui signifie que le chemin le plus rapide est aussi le plus sûr. Dans d'autres, le chemin sécurisé est plus long, mais garantit un niveau de sécurité maximal. Cette comparaison illustre parfaitement l'apport de la Recherche Opérationnelle : elle permet de formaliser les priorités et de prendre des décisions rationnelles selon le contexte et le niveau de sensibilité des données.

## 6.8 Synthèse

L'étude par scénarios montre que le modèle de routage sécurisé est :

- adaptable au niveau de sensibilité des données ;
- robuste face aux attaques et aux défaillances ;
- pertinent pour des applications réelles en cybersécurité et cloud computing.

## 6.9 Lecture décisionnelle et apport de la Recherche Opérationnelle

À travers les différents scénarios étudiés, il apparaît clairement que le choix d'un chemin dans un réseau ne dépend pas uniquement de critères techniques, mais surtout du contexte et des priorités décisionnelles.

Lorsque les données sont peu sensibles, le modèle favorise naturellement la performance. À l'inverse, dans un contexte de données critiques, le même réseau conduit à des choix totalement différents, privilégiant la sécurité au détriment de la latence. Cela montre que l'algorithme ne fournit pas une solution unique, mais une solution adaptée à un objectif précis.

Cette approche illustre parfaitement le rôle de la Recherche Opérationnelle : il ne s'agit pas seulement de calculer un plus court chemin, mais de formaliser un problème réel, d'intégrer des contraintes et des priorités, puis d'aider à la prise de décision.

Ainsi, la Recherche Opérationnelle permet de transformer un problème complexe de cybersécurité ou de cloud computing en un modèle mathématique exploitable, offrant des solutions rationnelles, justifiées et adaptables aux situations réelles.

# 7 Simulation

Veuillez consulter ci-dessous le QR code (ou le lien) permettant l'accès au mini-projet ainsi qu'au rapport correspondant.

https://drive.google.com/drive/u/1/folders/1uYDofbnsXVsre-6BVLbZCJYJT_yWPUjn

# 8 Conclusion Générale

Ce projet de routage de données sensibles nous a permis d'appliquer concrètement les concepts de la théorie des graphes à une problématique réelle de cybersécurité. À travers la modélisation d'un réseau maillé et l'implémentation de l'algorithme de Dijkstra, nous avons démontré qu'il est possible de concilier deux exigences souvent contradictoires : la performance (latence minimale) et la sécurité (risque d'interception réduit).

## 8.1 Principaux Apports

L'analyse comparative entre le chemin "pure performance" et le chemin sécurisé a révélé que la prise en compte du facteur risque n'entraîne pas nécessairement une dégradation significative des performances. Cette approche multi-critères permet aux décideurs de faire des choix éclairés en ajustant les coefficients de pondération ($\alpha$ et $\beta$) selon le

niveau de criticité des données transitées.

Les différents scénarios étudiés ont démontré la flexibilité et l'adaptabilité du modèle proposé. Que ce soit pour des données peu sensibles nécessitant une transmission rapide, ou pour des informations classifiées Secret Défense exigeant une sécurité maximale, notre approche s'est révélée capable de fournir des solutions optimales adaptées à chaque contexte.

## 8.2 Compétences Développées

Ce mini-projet nous a permis de renforcer nos compétences en modélisation mathématique, en programmation Python (NetworkX, visualisation), et surtout en analyse de problématiques de sécurité des réseaux. La dimension managériale de l'analyse nous a également sensibilisés à l'importance de traduire des résultats techniques en recommandations opérationnelles.

Au-delà des aspects techniques, ce travail nous a appris à :

— Formuler un problème réel sous forme d'un modèle mathématique rigoureux
— Implémenter et valider des algorithmes d'optimisation
— Interpréter les résultats dans une perspective décisionnelle
— Communiquer efficacement des solutions techniques à des non-spécialistes

## 8.3 Apport de la Recherche Opérationnelle

En définitive, ce travail illustre parfaitement comment la recherche opérationnelle constitue un outil d'aide à la décision indispensable pour les ingénieurs en cybersécurité, confrontés à des architectures de plus en plus complexes où chaque choix de routage peut avoir des implications critiques sur la confidentialité et l'intégrité des données.

La Recherche Opérationnelle ne se limite pas au calcul d'un chemin optimal, elle permet de formaliser les priorités, d'intégrer des contraintes multiples et de proposer des solutions rationnelles, justifiées et adaptables aux situations réelles. Cette approche méthodique transforme des décisions subjectives en choix objectifs, basés sur des critères mesurables et vérifiables.

## 8.4 Perspectives et Extensions Futures

Ce projet ouvre la voie à plusieurs extensions possibles :

— **Dynamisme du réseau** : Intégration de paramètres variables en temps réel (congestion, pannes)
— **Multi-chemins** : Calcul de chemins alternatifs pour assurer la redondance et la résilience
— **Apprentissage automatique** : Ajustement dynamique des coefficients $\alpha$ et $\beta$ selon l'historique des menaces
— **Optimisation multi-objectifs** : Prise en compte simultanée de plus de deux critères (coût, bande passante, fiabilité)
— **Simulation d'attaques** : Modélisation de scénarios d'attaques sophistiquées

## 8.5 Conclusion Finale

Ce mini-projet démontre que la maîtrise des outils de Recherche Opérationnelle est essentielle pour tout ingénieur en Cybersécurité et Cloud Computing. Dans un contexte où les infrastructures deviennent de plus en plus distribuées et les menaces de plus en plus sophistiquées, la capacité à modéliser, analyser et optimiser les systèmes complexes constitue un avantage stratégique majeur.

Le travail réalisé prouve qu'il est possible de construire des solutions techniques robustes tout en gardant une vision managériale claire, permettant ainsi aux décideurs de faire des choix éclairés basés sur des critères objectifs et mesurables. Cette double compétence technique et décisionnelle est au cœur du métier d'ingénieur moderne.